

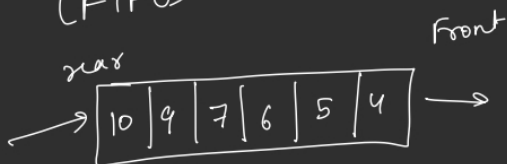
Queue Interface :

↳ Array } O(1)
↳ linked list

Stack

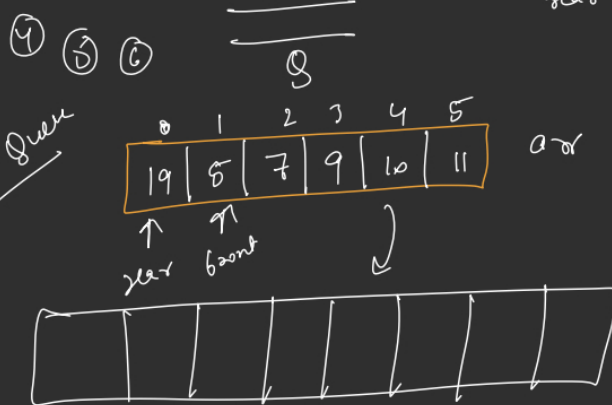
↳ Array } O(1)
↳ linked list

Array (FIFO)

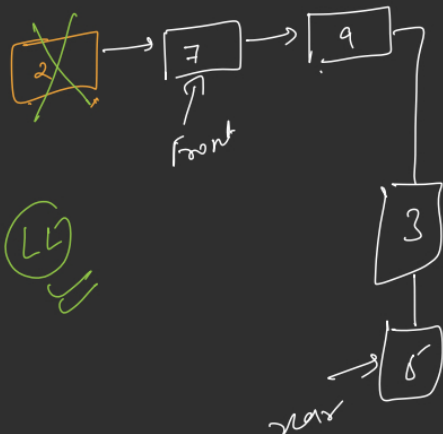


front = -1
rear = -1

→ resizing
→ Circular Queue

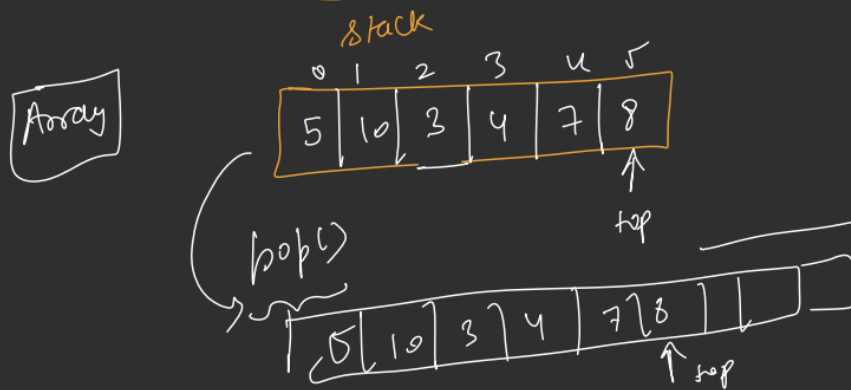
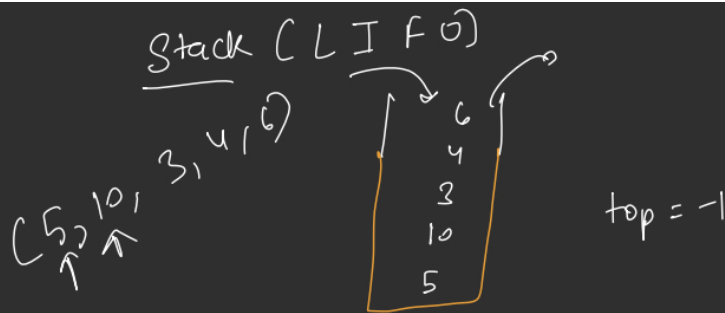


LL: (2, 7, 9, 3, 5)

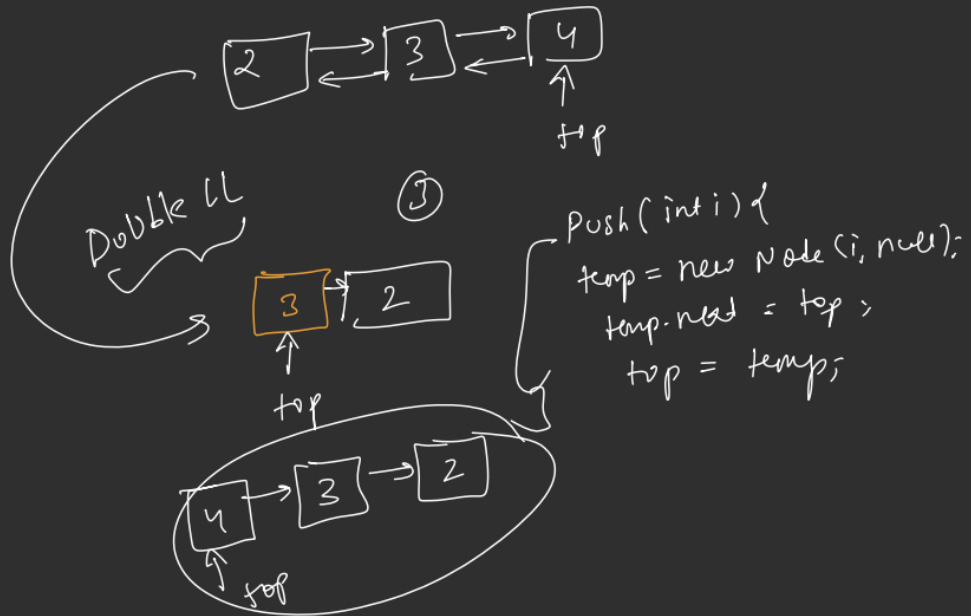


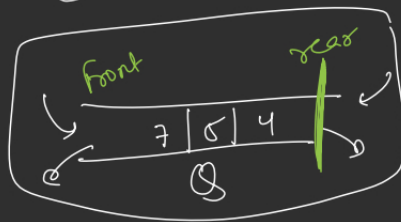
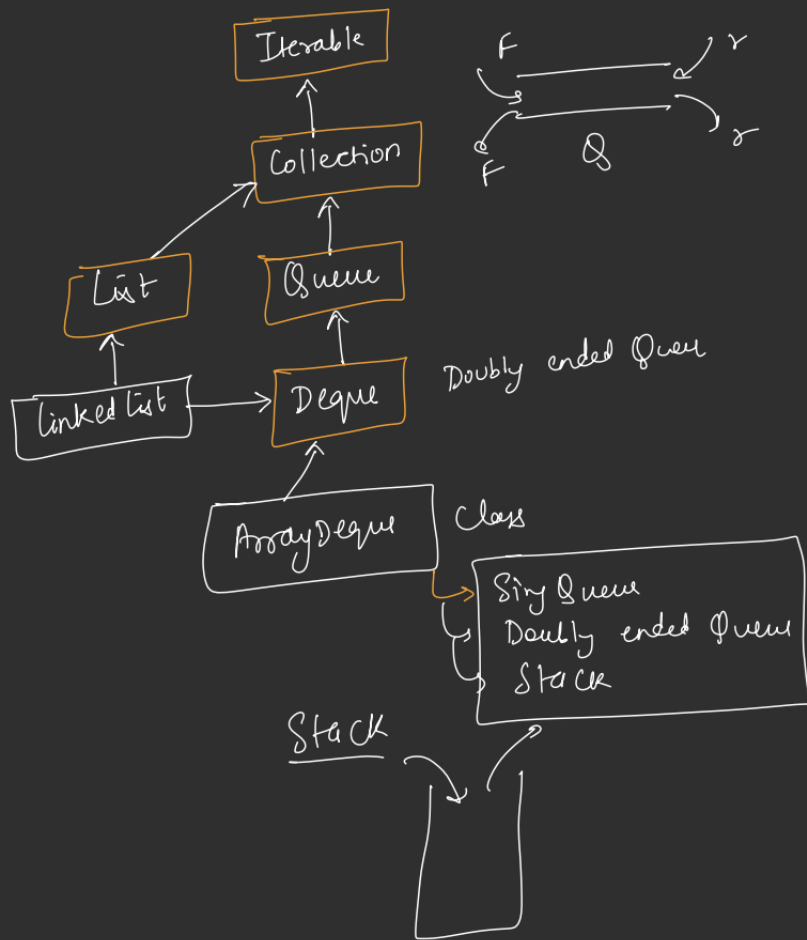
(LL)
✓

```
class Node {  
    int data;  
    Node next;  
}
```



LL → Single



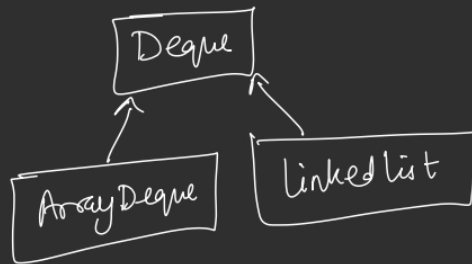


Stack

push()
pop()

obj.getFirst();
obj.getLast();

Internal Implementation



ArrayDeque

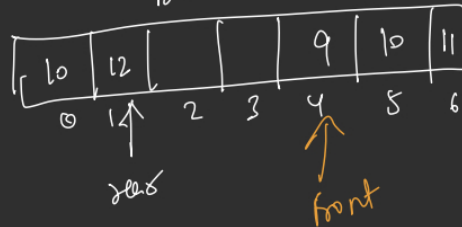
(5, 7, 9, 10, 11, 6)

Deque

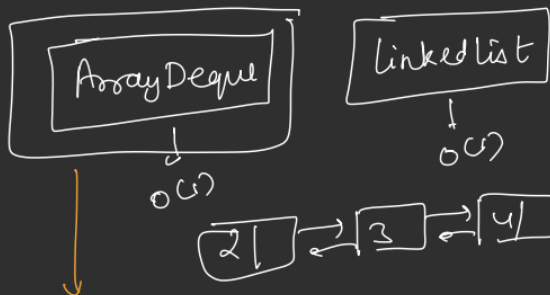
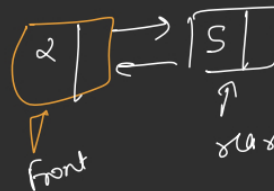
get()



↑ rear
↑ front

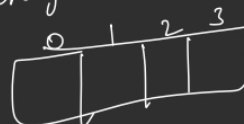


LinkedList (2, 5, 1)



Array

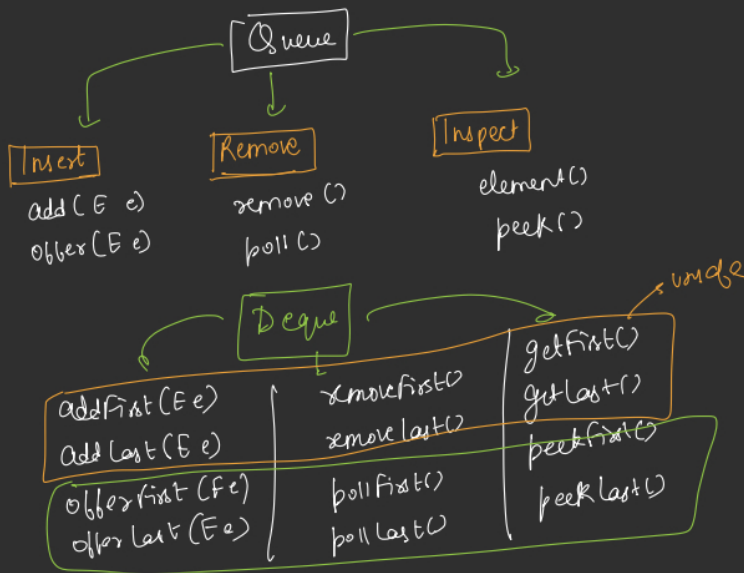
↳ Contiguous → Cache friendly



ArrayDeque → null (X)

LinkedList → null (✓)

Stack Legacy class
slow



Stack

offerFirst(E e)	addFirst(E e)
pollFirst()	removeFirst()
peekFirst()	getFirst()

push(E e)	→	offerFirst()
pop()	→	pollFirst()
peek()	→	peekFirst()

Stack → LIFO
Queue → FIFO

Behavioural Contract

Collection

→ remove(0).

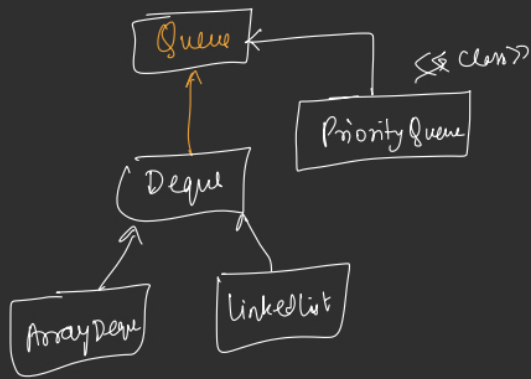


Stack

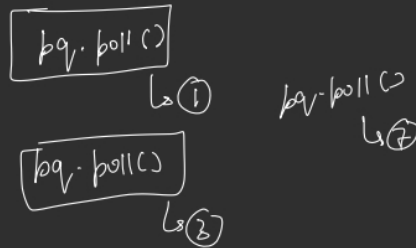
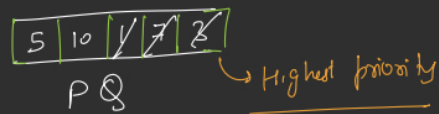
ArrayDeque

push() → offerFirst()

offerLast()



Priority Queue (FIFO) X



Internal Implementation

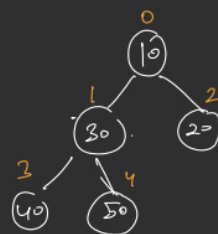
$\hookrightarrow$ Heap Data Structure

Binary Heap

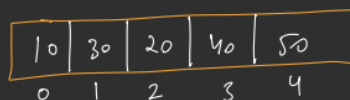
$\hookrightarrow$ Complete Binary Tree
 $\hookrightarrow$ implemented using stack //

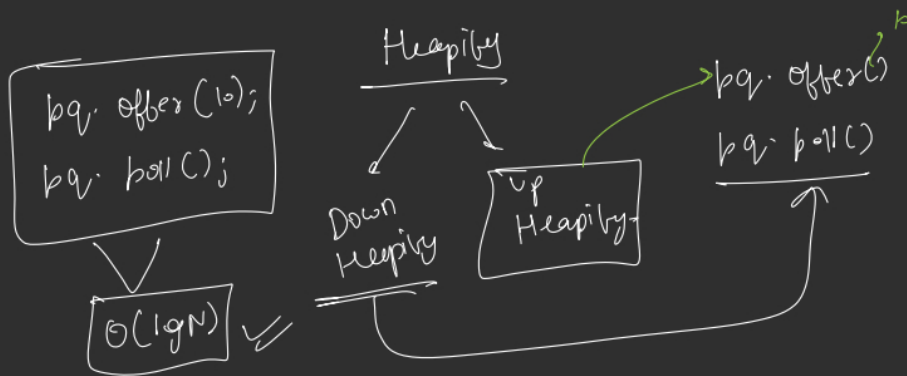
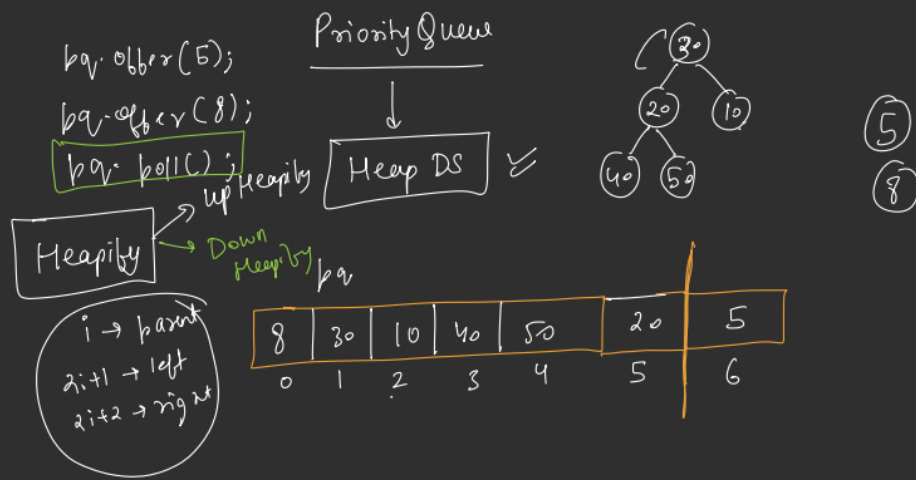
Complete Binary Tree

Heap DS
 Min Heap
 Max Heap



Parent $\rightarrow i$
 left $\rightarrow 2i+1$
 right $\rightarrow 2i+2$





Priority Queue		
add	remove	inspect
<code>add(E e)</code>	<code>remove()</code>	<code>element()</code>
<code>offer(E e)</code>	<code>poll()</code>	<code>peek()</code>

Safer alternatives